NAZARBAYEV UNIVERSITY

Programming for Engineers

Week 10: Understanding Program Efficiency I

Yanwei Wang
School of Engineering
Nazarbayev University

Fall 2026



Week 10 at a glance

Lecture 28

Measuring computational work
Input size, exact counts, early exits, and fair timing.

Lecture 29

Orders of growth and Big-O
Upper bounds, dominant terms, and combining costs.

Lecture 30

Recognizing complexity classes
Characteristic patterns and verified improvements.

Define the size $\rightarrow$ Count the work $\rightarrow$ Classify growth
Measure $\rightarrow$ Improve $\rightarrow$ Verify again

Course specification: Session 10, Understanding Program Efficiency I; CLOs 1 and 2.

Scope: the first efficiency unit

Week 10 core

- Measure computational work.
- Define growth and Big-O.
- Analyze characteristic classes.

Reserve deeper work

- Week 11: deeper efficiency analysis.
- Week 12: searching and sorting.
- Optional extensions are labelled.

Use what you already know

Use familiar loops, functions, collections, recursion, tests, and methods to explain computational cost.

Latest user-provided course outline; ENG 101 Course Specification, Sessions 10–12.

NAZARBAYEV UNIVERSITY

LECTURE 28

Measuring Computational Work

Define the problem size, count the work, and measure responsibly

NAZARBAYEV UNIVERSITY

Lecture 28 learning goals

1. Define the input size and the work being measured.
2. Distinguish operation counts, elapsed time, and auxiliary memory.
3. Trace full scans and early exits on known data.
4. Separate best, worst, and assumed average cases.
5. Collect a fair, repeatable timing experiment.
6. Explain what the evidence supports—and what it does not.

Central question

How does the work change when the engineering problem becomes larger?

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

Warm-up: fewer lines, less work?

Expression	What must Python do?
`readings[-1]`	Read one known position.
`sum(readings)`	Visit the entire numerical list.
`100.0 in readings`	Possibly inspect every element.

Predict before running

For a list of a million readings, which expression can still use a fixed amount of work? What information is missing for the membership test?

Individual, then pairs - 3 min

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

Why scale changes the engineering decision

A prototype

Ten measurements, four design choices, one model run.

The intended application

Millions of measurements, repeated design studies, or a deadline for each update.

Two correct methods can scale differently

A one-pass calculation grows with the number of readings. Comparing every reading with every other grows much faster.

Correctness remains first

A faster result is not useful if the units, interpretation, or required output have changed.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

Define the input size before any formula

Task	A useful size variable
Process a sensor log	$n =$ number of readings.
Compare two collections	n and $m =$ their separate lengths.
Visit a square grid	$q =$ points per side; $N = q^2$ total points.
Refine a numerical interval	$L_0/\varepsilon_x =$ required refinement ratio.

State it aloud

The same work can be quadratic in points per side but linear in the total number of grid points.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

Time and memory answer different questions

Time complexity

How does executed work grow as input size grows? Count comparisons, updates, or other specified operations.

Auxiliary space

How much extra storage is needed beyond the supplied input? Exclude input storage only when that convention is stated.

Example: a sensor summary

A running sum reads all n values but keeps a fixed number of scalars. A new list of n converted values also takes time to fill and storage to hold.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

Declare a simple cost model

- Count a comparison, list-index access, or fixed-size arithmetic operation as constant work.
- Count how often the chosen operation actually executes.
- Include work inside called functions and methods.
- Use bounded-size numeric values for the classroom comparisons.

A model, not a literal CPU instruction count

Python integers can grow in size. Copying a list or comparing long strings is not automatically one constant-time operation.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

Count work and measure time: use both

Operation count

A repeatable explanation of scaling. It exposes why a loop becomes expensive.

Elapsed time

An observation for a particular program, input, computer, and environment.

They are complementary

An operation-count model explains growth. Timing checks practical performance at tested sizes. Neither replaces correctness tests.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

Count every threshold comparison

```
def count_warnings(readings_c, limit_c):
    """Return warning count and number of threshold tests."""
    warnings = 0
    checks = 0
    for value_c in readings_c:
        checks += 1  # Count every comparison, not just a warning.
        if value_c > limit_c:
            warnings += 1
    return warnings, checks
# Exactly 30 deg C is permitted by this teaching rule.
print(count_warnings([19.0, 30.0, 31.0, 35.0], 30.0))
print(count_warnings([], 30.0))  # Empty input: zero comparisons.
```

What is counted?

One comparison per reading. A warning is an output event, not the complete measure of work.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

Trace the counter separately from the result

Reading (deg C)	Above 30?	Warnings so far	Checks so far
19.0	False	0	1
30.0	False	0	2
31.0	True	1	3
35.0	True	2	4

For any list of length n

This full scan performs exactly n threshold tests, even when the warning count is zero.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

Zero, one, and many inputs still matter

Input	Returned (warnings, checks)
`[]`	`(0, 0)`
`[30.0]`	`(0, 1)`
`[31.0]`	`(1, 1)`
n finite readings	Exactly n threshold tests.

Exact count versus total runtime

At $n = 0$ there are no threshold comparisons, but function entry, initialization, and returning still require work.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

Early exit changes the amount of work

```
def first_warning(readings_c, limit_c):
    """Return the first warning index (or None) and test count."""
    checks = 0
    for index, value_c in enumerate(readings_c):
        checks += 1  # Include the successful comparison.
        if value_c > limit_c:
            return index, checks  # Stop at the first warning.
    return None, checks
```

The output requirement changed

Return the first warning, not the total number of warnings. Stopping early is correct only for that requirement.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

Same length, different execution paths

Case for $n \geq 1$	Threshold comparisons
First reading exceeds the limit	1
First warning at index k	$k + 1$
Only the final reading warns	n
No warning exists	n

Two envelopes

The best case is the smallest work over valid inputs of size n. The worst case is the largest.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

An average requires an input assumption

A deliberately restricted model

Assume a warning exists and its first position is equally likely to be any of the n positions.

$$C_{\text{avg}}(n) = \frac{1 + 2 + \cdots + n}{n} = \frac{n+1}{2}.$$

Do not silently apply it to all sensor logs

A real log may have no warning, clustered events, or a position distribution that is not uniform.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

One line can still scan a million values

```
# One source line may hide work over all n elements.
readings = [2.0, 4.0, 6.0, 8.0]
print(len(readings))    # Read the stored list length.
print(readings[-1])     # Access one known position.
print(sum(readings))    # Visit every value to form a total.
print(9.0 in readings)  # An absent target requires a full
    scan.
```

Observed output

```
4
8.0
20.0
False
```

Count the operation, not the punctuation

The type and behavior of a function determine its work. A compact expression can hide a full loop.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

Measure a duration with perf_counter

```python
from time import perf_counter
# Build the input BEFORE starting this calculation-only timer.
values = [1.0] * 100_000
start_s = perf_counter()
result = sum(values)  # Time this calculation, not the printing.
elapsed_s = perf_counter() - start_s
assert result == 100_000.0  # Check correctness outside timing.
print(f"Elapsed: {elapsed_s:.6f} s")  # Hardware-dependent.
```

A duration is a difference

Subtract the earlier counter reading from the later reading. Printing and input creation are outside this measurement.

Python standard-library documentation: time.perf_counter and timeit; Wang, Section 10.21.

NAZARBAYEV UNIVERSITY

Repeat the experiment before concluding

```
from time import perf_counter
values = [1.0] * 100_000  # Setup is outside the timed region.
sum(values)  # One warm-up; not one of the recorded trials.
durations_s = []
for trial in range(5):
    start_s = perf_counter()
    result = sum(values)
    durations_s.append(perf_counter() - start_s)
    assert result == 100_000.0  # Do not time this check.
print("Trial times (s):", durations_s)
print("Best observed (s):", min(durations_s))
```

Report the chosen summary honestly

The minimum is the best observed trial, not a worst-case guarantee. Keep the raw trials so variability remains visible.

Python standard-library documentation: time.perf_counter and timeit; Wang, Section 10.21.

NAZARBAYEV UNIVERSITY

A fair comparison keeps the question fixed

- Use the same required result and equivalent input data.
- Exclude setup only if you are comparing calculation-only work.
- Keep printing, plotting, and assertions outside the timed region.
- Repeat trials at several input sizes on the same environment.
- Do not compare an already-mutated input with a fresh input.

State exclusions

Input creation and preprocessing still cost time in the full application; this benchmark excludes them on purpose.

Python standard-library documentation: time.perf_counter and timeit; Wang, Section 10.21.

NAZARBAYEV UNIVERSITY

A recorded experiment: summing float values

n	Best (ms/call)	Median (ms/call)	Trials $\times$ calls
1,000	0.0069	0.0069	5 $\times$ 20
10,000	0.0698	0.0713	5 $\times$ 20
100,000	0.7008	0.7134	5 $\times$ 20
1,000,000	7.2199	7.4085	5 $\times$ 20

Measurement protocol

n copies of `1.0`; one warm-up; five batches of 20 calls. Setup, checks, and output are excluded.

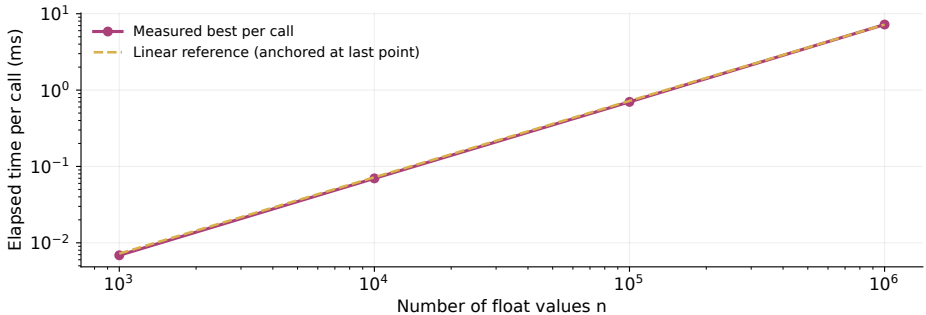
CPython 3.13.5; Linux x86_64.

These are measurements from one run. Raw trials are included in the source package.

Original benchmark supplied with this package; timer semantics: Python documentation. No hardware-independent speed claim.

NAZARBAYEV UNIVERSITY

Read the timing trend, not just one number



Both axes are logarithmic. The dashed line is a linear reference anchored at the final measured point. Agreement over these sizes supports—but does not prove—linear growth.

Original measured data and reference line; data/timing.csv and tools/build_figures.py.

NAZARBAYEV UNIVERSITY

A scaling ratio is a useful diagnostic

$$R(n) = \frac{T(2n)}{T(n)}.$$

Dominant model	Expected large-n trend
$T(n) = a$	Ratio near 1
$T(n) = an + b$	Ratio approaches 2
$T(n) = an^2 + bn + c$	Ratio approaches 4

A diagnostic, not a proof

Small inputs, noise, caches, and setup costs can distort timing ratios. Use the ratio with the algorithmic count.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

Activity: design a reproducible measurement

Task

Compare `sum(values)` with a Python loop that accumulates every value.

1. Choose three increasing list lengths.
2. Write a known-answer test before timing.
3. State exactly which region will be timed.
4. Describe how you will repeat and summarize the trials.

Pairs - 4 min

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

A method call does not remove hidden work

```
class SensorLog:
    """A familiar class does not make a scan constant-time."""
    def __init__(self, readings_c):
        self.readings_c = readings_c.copy()  # Copy n references.
    def count_high(self, limit_c):
        count = 0
        for value_c in self.readings_c:  # n comparisons per call.
            if value_c > limit_c:
                count += 1
        return count
```

Connect to Weeks 8–9

Constructing the log copies n references. Each `count_high()` call scans n readings. A short interface can hide substantial work.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101. Connection to the supplied Week 8 sensor examples.

NAZARBAYEV UNIVERSITY

Before writing an efficiency claim

1. What does n represent?
2. What is the required output and input contract?
3. Which operations are counted?
4. Which input case is analyzed?
5. What setup or storage is excluded?
6. What did measurement actually establish?

A stronger sentence

For n finite readings, this function checks every reading once and retains only a fixed number of running scalars.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

Lecture 28 exit ticket

New situation

A pressure log contains n readings. A function returns the first reading strictly above 200 kPa, or `None`.

1. State the best- and worst-case comparison counts for $n \geq 1$.
2. Explain why a three-line function need not be constant work.
3. Name one quantity that must be held fixed in a timing comparison.

Individual - 3 min

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101.

NAZARBAYEV UNIVERSITY

LECTURE 29

Orders of Growth and Big-O

Turn execution counts into clear statements about scaling

NAZARBAYEV UNIVERSITY

Lecture 29 learning goals

1. Separate an exact operation count from its order of growth.
2. Explain Big-O as an eventual upper bound.
3. Use a tight growth description without confusing it with a case.
4. Analyze consecutive, nested, and conditional work.
5. Recognize constant bounds and independent dimensions.
6. Keep practical constants visible when comparing small inputs.

Central question

Which term controls the growth as the problem becomes large?

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

NAZARBAYEV UNIVERSITY

Warm-up: which term will eventually dominate?

$$A(n) = 1000, \qquad B(n) = n, \qquad C(n) = 2n^2.$$

n	$A(n)$	$B(n)$	$C(n)$
10	1,000	10	200
100	1,000	100	20,000
1,000	1,000	1,000	2,000,000

Predict the next row - 3 min

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

NAZARBAYEV UNIVERSITY

Count three stages using one declared operation

```
def selected_additions(n):
    """Count only the executed += 1 additions; n >= 0."""
    count = 0
    for _ in range(1000):
        count += 1  # A fixed amount, independent of n.
    for _ in range(n):
        count += 1  # n more selected additions.
    for i in range(n):
        for j in range(n):
            count += 1  # Two additions per pair.
            count += 1
    return count
print(selected_additions(10))  # 1000 + 10 + 2 * 100.
```

$$C(n) = 1000 + n + 2n^2.$$

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8. Adapted counted-stage structure; selected additions only.

NAZARBAYEV UNIVERSITY

Keep the dominant growth term

Exact selected count	For large n	Growth description
$1000 + n + 2n^2$	Quadratic term dominates	$O(n^2)$
$7n + 30$	Linear term dominates	$O(n)$
12	No size dependence	$O(1)$

Dropping is not deleting

The constants and lower-order terms are still executed. Asymptotic notation intentionally summarizes their long-run effect.

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

NAZARBAYEV UNIVERSITY

An order of growth is not a stopwatch reading

It says

How a cost function behaves as the chosen input size becomes sufficiently large.

It does not say

An exact number of seconds, the fastest implementation for every size, or the correctness of the output.

Use both kinds of evidence

Keep the growth argument and the measured runtime. They answer different engineering questions.

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

Big-O: an eventual upper bound

$$T(n) \in O(g(n)) \quad \text{when} \quad 0 \le T(n) \le Cg(n) \quad (n \ge n_0)$$

What the symbols mean

$C > 0$ is a fixed multiplier. n_0 is a fixed starting size. Both must be independent of n.

Read it in words

After a sufficiently large input size, a constant multiple of $g(n)$ bounds the work from above.

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

A complete small bound argument

$$T(n) = 3n + 20.$$

$$n \geq 20 \implies 20 \leq n \implies 3n + 20 \leq 4n.$$

Choose witnesses

$C = 4$ and $n_0 = 20$ establish $T(n) \in O(n)$.

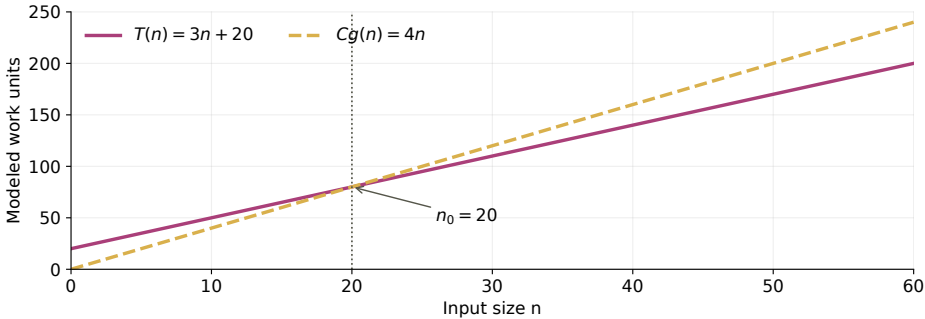
Why not only test a few values?

Finite tests can illustrate an inequality. The algebra establishes it for every $n \geq 20$.

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

NAZARBAYEV UNIVERSITY

See the bound after the threshold



Below n_0 is allowed. The upper-envelope inequality must hold for all sufficiently large inputs, not necessarily for every tiny input.

Original figure of the inequality in Wang, Appendix G.7.

NAZARBAYEV UNIVERSITY

Big-O and tight growth are not identical

Statement	Interpretation
$T(n) \in O(n)$	An asymptotic upper bound.
$T(n) \in \Theta(n)$	Matching upper and lower linear bounds.
$T(n) = 3n + 20$	Also in $O(n^2)$, but that bound is loose.

Our classroom convention

Give the most informative simple bound justified by the count. Use Θ when explicitly claiming a tight order.

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

NAZARBAYEV UNIVERSITY

The case and the bound are separate choices

Choose the cost function

Best-case work, worst-case work, or expected work under stated probabilities.

Then bound that function

Big-O is an upper-bound relation. It can describe any one of those chosen cost functions.

Example: first-warning scan

Best case: $\Theta(1)$. Worst case: $\Theta(n)$. Conditional uniform-position average: $\Theta(n)$.

Do not memorize a false identity

Big-O does not mean worst case by definition.

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

NAZARBAYEV UNIVERSITY

Activity: simplify without losing the meaning

Count model	Your most informative simple class
$8n + 200$	_____
$3n^2 + 7n + 5$	_____
$50 + 4\log_2 n$	_____
$n\log_2 n + 10n$	_____

Pairs - 3 min

Explain one answer

Identify the dominant term and the assumption that constants stay fixed.

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

NAZARBAYEV UNIVERSITY

Consecutive stages add their work

```
def two_passes(values):
    """Count visits in two consecutive full passes."""
    visits = 0
    for value in values:
        visits += 1  # First pass: n visits.
    for value in values:
        visits += 1  # Second pass: n visits.
    return visits
print(two_passes([2, 4, 6, 8]))  # 2n, not n squared.
```

Observed output

```
8
```

$$C(n) = n + n = 2n \in \Theta(n).$$

Reason from execution

The second pass starts after the first finishes. It is not repeated inside every first-pass iteration.

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

NAZARBAYEV UNIVERSITY

Independent nested ranges multiply

```
def design_pairs(diameters_mm, speeds_rpm):
    """Count candidate pairs; no engineering model is solved."""
    evaluations = 0
    for diameter_mm in diameters_mm:
        for speed_rpm in speeds_rpm:
            evaluations += 1  # One candidate pair considered.
    return evaluations
# n = 3 diameter options; m = 4 speed options.
print(design_pairs([6, 8, 10], [500, 1000, 1500, 2000]))
```

Observed output

```
12
```

$$C(n, m) = \underbrace{n}_{\text{diameters}} \times \underbrace{m}_{\text{speeds}}.$$

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

NAZARBAYEV UNIVERSITY

Two dimensions must not be collapsed silently

How the ranges grow	Candidate count	Growth
n diameters, m speeds	nm	$\Theta(nm)$
n diameters, 4 fixed speeds	$4n$	$\Theta(n)$
n options on both axes	n^2	$\Theta(n^2)$

A model evaluation may add another factor

If each candidate uses N_t time steps, the work may scale as nmN_t under a fixed-cost time-step model.

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8. Wang, Appendix G.3.2 and G.30.

NAZARBAYEV UNIVERSITY

Two loops do not automatically mean quadratic

```
def repeated_checks(values):
    """Three fixed checks per value: exactly 3n visits."""
    checks = 0
    for value in values:
        for repeat in range(3):  # This bound does not grow.
            checks += 1
    return checks
print(repeated_checks([1, 2, 3, 4]))
```

Observed output

```
12
```

The inner bound is fixed

Three checks for each of n inputs: $3n$ checks, so $\Theta(n)$ selected work.

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

Dependent bounds need a sum

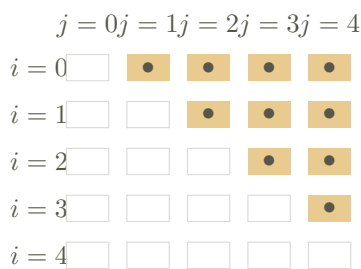
```
def unique_pair_count(n):
    """Count unordered pairs of distinct indices; n >= 0."""
    pairs = 0
    for i in range(n):
        for j in range(i + 1, n):  # Never compare i with itself.
            pairs += 1  # Never count the reverse pair again.
    return pairs
for n in [0, 1, 4, 8]:
    print(n, unique_pair_count(n))
```

$$C(n) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}.$$

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

NAZARBAYEV UNIVERSITY

A triangular grid explains the count



For $n = 5$:
$4 + 3 + 2 + 1 + 0 = 10$ pairs.

The shaded cells satisfy $j > i$.

A factor of one-half does not change the order

$n(n-1)/2$ and n^2 have the same quadratic order, but they are not the same exact count.

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

NAZARBAYEV UNIVERSITY

A called operation can hide another scan

```
def approved_count(installed_ids, approved_ids):
    """Both arguments are lists of fixed-size identifiers."""
    count = 0
    for sensor_id in installed_ids:
        # Membership can scan all m approved identifiers.
        if sensor_id in approved_ids:
            count += 1
    return count
print(approved_count([1, 4, 7], [1, 2, 3, 4]))
```

Worst-case argument

For n installed IDs and a list of m approved IDs, each membership test can inspect m entries: $O(nm)$ work.

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

NAZARBAYEV UNIVERSITY

Conditionals: analyze the reachable routes

```
def review_work(n, full_review):
    """Count selected actions for either permitted route."""
    actions = 0
    if full_review:
        for i in range(n):
            for j in range(n):
                actions += 1  # Full route: n squared.
    else:
        for i in range(n):
            actions += 1  # Quick route: n.
    return actions
print(review_work(4, False), review_work(4, True))
```

Observed output

```
4 16
```

For growing n

Quick route: linear. Full route: quadratic. With either permitted, the worst-case cost is quadratic.

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

Constants can reverse a small-input ranking

$$T_A(n) = 100n, \qquad T_B(n) = n^2.$$

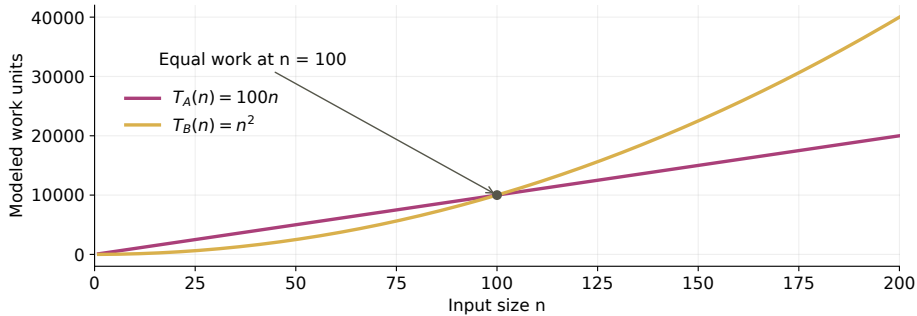
n	$T_A(n)$	$T_B(n)$	**Less modeled work**
10	1,000	100	B
100	10,000	10,000	Equal
1,000	100,000	1,000,000	A

No universal winner from one tiny run

The linear model eventually wins, but not at every input size.

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

The crossover is a model-specific quantity



Read the axes. The vertical axis is modeled work, not measured seconds. The crossover at $n = 100$ follows from the two stated formulas.

Original graph of the declared models; conceptual basis: Wang, G.5 and G.7.

NAZARBAYEV UNIVERSITY

Logarithm bases differ only by a constant

$$\log_2 n = \frac{\log_{10} n}{\log_{10} 2}.$$

For growth classification

The denominator is a fixed constant, so $O(\log_2 n) = O(\log_{10} n) = O(\log n)$.

For an exact count

Keep the base. Halving suggests base 2; removing decimal digits suggests base 10.

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

NAZARBAYEV UNIVERSITY

Debug the explanation, not only the program

Incorrect explanation	Question that repairs it
Two consecutive loops are quadratic.	Do they run one after another or inside each other?
Two nested loops are always quadratic.	Does the inner bound grow with the input?
A built-in call costs one operation.	What does that call do internally?
Big-O gives the exact worst-case time.	Which case, cost model, and kind of bound?

Explain one correction - 3 min

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

NAZARBAYEV UNIVERSITY

Lecture 29 exit ticket

$$C(n) = 5n^2 + 40n + 9.$$

1. Give the most informative simple Big-O bound.
2. Explain why $O(n^3)$ is also valid but less informative.
3. A loop makes 8 fixed checks for every reading. State its growth.
4. Two independent lists have lengths n and m. State the all-pairs count.

Individual - 3 min

Wang, Appendix G.6–G.9; Guttag, Section 9.2. Explicit bound terminology follows Wang, G.7–G.8.

NAZARBAYEV UNIVERSITY

LECTURE 30

Recognizing Complexity Classes

Connect code patterns to engineering choices at larger scales

NAZARBAYEV UNIVERSITY

Lecture 30 learning goals

1. Recognize constant, logarithmic, linear, and log-linear patterns.
2. Count quadratic and cubic work with explicit size definitions.
3. Recognize exponential branching without running enormous cases.
4. Explain why recursion alone does not determine complexity.
5. Separate time, retained output, and auxiliary state.
6. Improve repeated work while preserving the required result.

Central question

Which algorithmic structure explains the growth class?

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

A first map of the growth classes

Class	Characteristic pattern	Meaning of n
$O(1)$	One indexed lookup	List length
$O(\log n)$	Repeatedly halve remaining size	Initial size
$O(n)$	One full scan	Readings
$O(n \log n)$	n tasks, each with log stages	Items
$O(n^2)$, $O(n^3)$	Two- or three-dimensional full grids	Points per side
$O(2^n)$	Visit all binary-choice combinations	Independent choices

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

Constant work: retrieve a known position

```
def latest_reading(readings):
    """Return the last stored reading; reject an empty list.
     """
    if not readings:
        raise ValueError("No readings available")
    return readings[-1]  # One indexed access, regardless of
      n.
print(latest_reading([20.0, 22.0, 25.0]))
```

Observed output

```
25.0
```

Why $O(1)$ here?

The number of operations does not grow with the length of the existing list. It is not zero time.

Input creation and full element validation are not part of this function.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Logarithmic work: reduce by a fixed factor

```
def halving_steps(size):
    """Return the number of floor-halvings; integer size >= 1."""
    steps = 0
    while size > 1:
        size //= 2  # Integer division shrinks the problem.
        steps += 1
    return steps
for size in [1, 8, 16, 32, 1024]:
    print(size, halving_steps(size))
```

For powers of two

If $n = 2^k$, the loop makes k halvings. Doubling n adds one iteration.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Trace shrinking size instead of counting lines



$$\frac{n}{2^k} = 1 \quad \Longrightarrow \quad k = \log_2 n \qquad (n = 2^k).$$

One extra input doubling

64 needs six halvings, not ten or thirty-two. The update rule creates the logarithm.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Week 3 revisited: the cost of interval refinement

$$L_k = \frac{L_0}{2^k}, \qquad L_k \le \varepsilon_x \implies k \ge \left\lceil \log_2\left(\frac{L_0}{\varepsilon_x}\right) \right\rceil .$$

Example

Reduce a bracket of width 8 s to at most 0.125 s: $8/0.125 = 64$, so six halvings suffice.

Specify both the target and the model

This is an interval-width count. A residual-only stopping rule, model evaluation cost, or floating-point stagnation needs separate analysis.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples. Connection to the supplied Week 3 bisection lecture; width-based count is explicitly distinguished.

NAZARBAYEV UNIVERSITY

Linear work: integrate a sampled power record

```
def total_energy_j(power_w, dt_s):
    """Sum power * duration; fixed step in s, power in W."""
    energy_j = 0.0
    for value_w in power_w:
        energy_j += value_w * dt_s  # One update per sample.
    return energy_j
# Piecewise-constant power: (10 + 20 + 30) W * 2 s.
print(total_energy_j([10.0, 20.0, 30.0], 2.0))
```

Observed output

```
120.0
```

Model: $E \approx \sum_{i=1}^{n} P_i \, \Delta t$.

Count and units

One multiplication and accumulation per reading. W times s gives J. The piecewise-constant time-step model is explicit.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Log-linear work: n items times log stages

```
def staged_work(n):
    """A counting model, NOT a sorting algorithm; n >= 1."""
    work = 0
    for item in range(n):
        size = n  # Reset: each item receives all halving stages.
        while size > 1:
            size //= 2
            work += 1
    return work  # n * floor(log2(n)) selected increments.
for n in [8, 16, 32]:
    print(n, staged_work(n))
```

Count the selected increments

$C(n) = n\lfloor \log_2 n \rfloor$, giving $\Theta(n \log n)$ for growing n.

This counting model is not a sorting implementation. Sorting algorithms are taught in Week 12.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Quadratic work: compare every pair

```
def maximum_gap(values):
    """Return max |a - b|; assume a nonempty finite list."""
    largest_gap = 0.0
    checks = 0
    for a in values:
        for b in values:
            checks += 1  # Includes self-pairs and reverse pairs.
            largest_gap = max(largest_gap, abs(a - b))
    return largest_gap, checks
print(maximum_gap([20.0, 23.0, 25.0]))
```

Why n^2?

Every one of n first readings is compared with every one of n second readings. Doubling n gives four times as many pair checks.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Cubic work: visit a three-dimensional grid

```
def grid_locations(n):
    """Count one visit per point in an n by n by n grid."""
    visits = 0
    for i in range(n):
        for j in range(n):
            for k in range(n):
                visits += 1  # Count only; do not store the
    grid.
    return visits
print(grid_locations(4))  # 64 points.
print(grid_locations(8))  # 512 points: eight times as many.
```

Observed output

```
64
512
```

Define the size carefully

There are n^3 points for n points per side. In terms of the total point count $N = n^3$, visiting each point is linear in N.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Polynomial growth: the exponent is fixed

$$n, \quad n^2, \quad n^3, \quad n^p \text{ for fixed } p.$$

Scaling change	One pass	Square grid	Cube grid
n doubles	$\times 2$	$\times 4$	$\times 8$
n multiplies by 10	$\times 10$	$\times 100$	$\times 1000$

Polynomial is not automatically practical

A large exponent, expensive model call, or large input can still make the computation infeasible.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Exponential work: visit every binary choice

```
def visit_binary_choices(n):
    """Return (leaves, calls) for n binary choices; use small n."""
    if n == 0:
        return 1, 1  # One complete configuration and one call.
    left, left_calls = visit_binary_choices(n - 1)
    right, right_calls = visit_binary_choices(n - 1)
    # Count both subtrees and this current call.
    return left + right, 1 + left_calls + right_calls
print(visit_binary_choices(4))  # 16 leaves, 31 total calls.
```

$$\text{leaves} = 2^n, \qquad \text{total calls} = 1 + 2 + \cdots + 2^n = 2^{n+1} - 1.$$

Do not confuse a count with an implementation

This code actually visits both subtrees. Merely evaluating `2**n` does not enumerate 2^n configurations.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Fibonacci revisited: repeated subproblems

```
def fibonacci_counted(n):
    """Return (F(n), calls); keep Week 6's F(0)=F(1)=1."""
    if n == 0 or n == 1:
        return 1, 1  # One base-case call.
    a, ca = fibonacci_counted(n - 1)
    b, cb = fibonacci_counted(n - 2)
    return a + b, 1 + ca + cb  # Include this call in the total.
for n in [5, 10, 20]:
    print(n, fibonacci_counted(n))  # Do not try very large n.
```

A useful upper bound, not a tight one

Two smaller recursive calls can create exponentially many calls. $O(2^n)$ is a convenient upper bound for this naive implementation.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

An iterative alternative avoids recomputation

```
def fibonacci_iterative(n):
    """Return (F(n), additions) with F(0)=F(1)=1."""
    previous, current = 1, 1
    additions = 0
    for index in range(2, n + 1):
        previous, current = current, previous + current
        additions += 1  # Retain only the two latest values.
    return current, additions
for n in [5, 10, 20]:
    print(n, fibonacci_iterative(n))
```

n	Recursive calls	Iterative additions
5	15	4
10	177	9
20	21,891	19

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Recursion is not a complexity class

```
def factorial(n):
    """Return n! for a small nonnegative integer n."""
    if n == 0:
        return 1
    return n * factorial(n - 1)  # ONE smaller call, not two.
print(factorial(5))  # n multiplications; n + 1 active frames.
# This is linear in n under the stated unit-cost arithmetic model.
# The numeric output n! does NOT imply factorial running time.
```

Follow the call structure

One call per level gives $n + 1$ frames and n multiplications. This factorial calculation is linear in n under unit-cost arithmetic.

Output size is not runtime

Returning the number $n!$ does not mean the program takes $n!$ steps.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Put normalized growth models side by side

Model	$n = 10$	$n = 20$	$n = 40$
$\log_2 n$	3.32	4.32	5.32
n	10	20	40
$n \log_2 n$	33.2	86.4	212.9
n^2	100	400	1,600
n^3	1,000	8,000	64,000
2^n	1,024	1,048,576	1.10×10^{12}

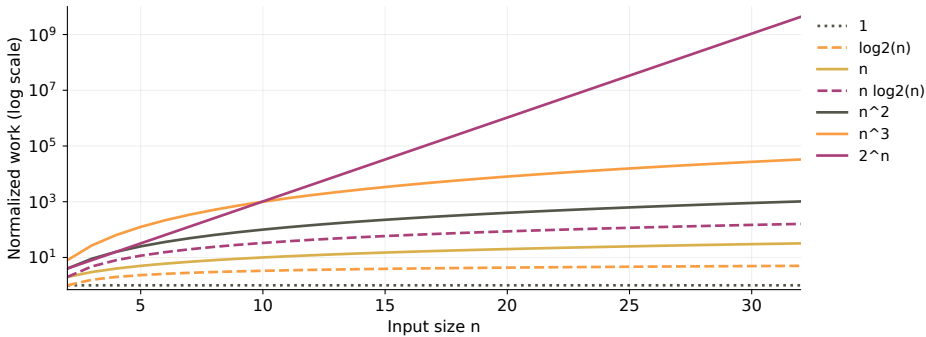
These are mathematical model values

Each leading coefficient is set to 1. They are not measured instructions or predicted seconds.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Growth rates separate dramatically



Normalized mathematical models, not benchmark timings. The vertical axis is logarithmic.

Original plot from the displayed models; tools/build_figures.py. Concepts: Wang G.10; Guttag9.3.

NAZARBAYEV UNIVERSITY

Activity: work within a declared budget

Hypothetical budget

An analysis allows at most 10^6 normalized work units. Assume the exact cost model is $C(n) = n^2$.

1. What is the largest integer n allowed?
2. If n doubles, by what factor must the budget grow?
3. Would knowing only $C(n) \in O(n^2)$ determine the same numerical limit?

Pairs - 4 min

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Time and retained storage can change separately

Task over n values	Time	Extra retained storage
Return a final running total	$O(n)$	$O(1)$ scalars
Return all cumulative totals	$O(n)$	$O(n)$ output values
Recursive factorial	$O(n)$*	$O(n)$ call frames*

State the counting convention

Input storage is excluded. Retained output is included here. *Factorial uses the unit-cost / scalar-state model; large integers need more bits.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Cumulative totals: repeated work

```
def prefix_totals_repeated(values):
    """Return totals through each sample without mutating
     input."""
    totals = []
    for end in range(1, len(values) + 1):
        # Slice and rescan lengths 1, 2, ..., n.
        totals.append(sum(values[:end]))
    return totals
print(prefix_totals_repeated([2.0, 3.0, 4.0]))
```

Observed output

```
[2.0, 5.0, 9.0]
```

The repeated work is hidden

The sliced lengths are $1, 2, \ldots, n$. Copying and summing them repeatedly produces quadratic total work.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Reuse the subtotal instead of recomputing it

```
def prefix_totals_running(values):
    """Return cumulative totals in one pass; preserve the
     input."""
    totals = []
    running = 0.0
    for value in values:
        running += value  # Reuse the previous subtotal.
        totals.append(running)
    return totals
print(prefix_totals_running([2.0, 3.0, 4.0]))
```

Observed output

```
[2.0, 5.0, 9.0]
```

Element visits: $C_{\text{repeated}} = n(n+1)/2$; $C_{\text{running}} = n$.

Preserve the contract

Both return every prefix total and leave the input unchanged. Test values, empty input, output length, and non-mutation.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Improve the work, then test and measure again

1. Keep the required output, order, units, and accuracy fixed.
2. Identify repeated scans or unnecessary copies.
3. Change the algorithm or data structure deliberately.
4. Rerun known-answer, boundary, and non-mutation tests.
5. Measure the new implementation on representative sizes.
6. Record the remaining time and storage trade-off.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples. Wang, Appendix G.32.

NAZARBAYEV UNIVERSITY

Quick poll: which statements need correction?

1. Every `while` loop is linear.
2. Every recursive function is exponential.
3. Two programs with the same Big-O have the same runtime.
4. A built-in call is always constant work.
5. A fast result is acceptable even when it fails the original tests.

Vote, then justify - 3 min

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Lecture 30 exit ticket

Pattern	State the class and one reason
One indexed lookup	_____
n independent fixed-cost checks	_____
Halve size until it reaches 1	_____
Visit all $nimesn$ pairs	_____
Visit all binary-choice leaves	_____

Individual - 3 min

One final sentence

Explain why the prefix-total improvement must be tested before its speed is trusted.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples.

NAZARBAYEV UNIVERSITY

Optional: exact loop counts include the final test

Code idea	Body passes	Condition evaluations
`while size > 1`	k	$k+1$
Floor-halve $n = 8$	3	4
Floor-halve $n = 10$	3	4
Already $n = 1$	0	1

For this exact update

With `size //= 2` and integer $n \geq 1$, body passes equal $\lfloor \log_2 n \rfloor$. Different rounding and stopping rules can require a ceiling.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101. Technical counting clarification.

NAZARBAYEV UNIVERSITY

Optional: a standard microbenchmark helper

```
from timeit import repeat
values = [1.0] * 100_000  # Input creation is not timed.
def task():
    # A zero-argument function lets timeit repeat the same task.
    return sum(values)
assert task() == 100_000.0
# Each of five trials executes task 20 times.
trials_s = repeat(task, repeat=5, number=20)
print("Best observed seconds per call:", min(trials_s) / 20)
```

Two different repetition counts

`repeat=5`: five timing trials. `number=20`: twenty task calls inside each trial. Divide the selected trial duration by 20.

Python standard-library documentation: time.perf_counter and timeit; Wang, Section 10.21.

NAZARBAYEV UNIVERSITY

Optional: what counts as input size?

Integer magnitude n

Counting down from n uses about n updates under a unit-cost arithmetic model.

Binary input length b

Representing a positive integer needs about $\log_2 n$ bits. A count proportional to n may be exponential in b.

Large integers are not fixed-size numbers

Their arithmetic and storage costs grow with bit length. Week 10 uses the simpler declared model for small numerical demonstrations.

Wang, Sections 10.20–10.22 and Appendix G.3–G.6; Guttag, Section 9.1. Teaching examples adapted for ENG 101. Wang G.3, G.4, and the declared unit-cost model.

NAZARBAYEV UNIVERSITY

Optional: familiar operations have different costs

Operation	Typical cost model	Important qualification
`len(a)`, `a[i]`	$O(1)$	For an existing Python list.
`a.copy()`, `sum(a)`	$O(n)$	Copy references / visit values.
`x in a`	$O(n)$ worst case	For list membership.
`a.append(x)`	$O(1)$ amortized	An individual resize may cost more.
`mapping[key]`	$O(1)$ average-case	Not a universal worst-case guarantee.

Conventional container costs assume bounded-cost element operations.

Wang, Appendix G, sections on Python lists, dictionaries, and amortized cost. Optional reference.

NAZARBAYEV UNIVERSITY

Reference: match each class to its structure

Growth	A reason, not just a name
$O(1)$	A fixed number of actions.
$O(\log n)$	A fixed-factor reduction at each stage.
$O(n)$	A fixed amount of work for every input item.
$O(n \log n)$	A full-size workload across logarithmically many stages.
$O(n^2),\ O(n^3)$	Two or three growing dimensions.
$O(2^n)$	Every binary-choice configuration is visited.
$O(n!)$	Every ordering of n distinct items is visited.

Wang, Sections 10.23–10.27 and Appendix G.10; Guttag, Section 9.3. Original counted examples. Wang G.10: factorial growth is an optional extension.

Practice before Week 11

1. Explain the exact count and growth of three consecutive scans.
2. Compare a fixed inner loop with a triangular inner loop.
3. Trace the logarithmic counter for 1, 10, and 64.
4. Test both prefix-total implementations on an empty and a signed dataset.
5. Write a four-sentence efficiency report: size, operation, case, and evidence.

Status of these tasks

Practice activities, not a new graded assignment. Use Moodle for any separately announced assessment instructions.

Instructor-designed practice aligned with the user-provided Week 10 outline; assessment details are not added.

Reading and course-source alignment

- **ENG 101 Course Specification:** Session 10; CLOs 1 and 2. Sessions 11–12 define the continuation.
- **Wang, Programming for Engineers with Python:** Sections 10.20–10.27; Appendix G.3–G.10 and G.32.
- **Guttag, Introduction to Computation and Programming Using Python, 2nd ed.:** Chapter 9, especially Sections 9.1–9.3.

How these slides use the sources

Terminology and topic sequence follow the course. Counted sensor, grid, and prefix examples are teaching adaptations. No textbook timing is reused as a current benchmark.

Course sources supplied by the instructor; new examples and diagrams are explicitly marked adaptations.

Technical references and the next two weeks

Python standard library

time.perf_counter: elapsed-time measurement.
timeit: repeated microbenchmarking.
Official documentation checked 14 September 2026.

Week 11: efficiency II

Deeper class comparisons, hidden work, and practical time–space trade-offs.

Week 12: searching and sorting

Implement and trace searches and sorts; examine their requirements and costs.

Week 10 establishes growth reasoning; the dedicated search/sort unit follows later.

User-provided Weeks 10–12 outline; Python standard-library documentation.

A correct answer must also be practical

Understand the work before optimizing it.

State the input size.

Explain the growth.

Measure the implementation.

Keep the engineering result correct.

Week 10 - Understanding Program Efficiency I